Merging Similar Patterns for Hardware Prefetching

Shizhi Jiang*, Qiusong Yang✉†, and Yiwei Ci‡
*University of Chinese Academy of Sciences
*†‡Institute of Software, Chinese Academy of Sciences
Beijing, China
Email: *shizhi@nfs.iscas.ac.cn, †qiusong@iscas.ac.cn, ‡yiwei@iscas.ac.cn

***Abstract*—One critical challenge of designing an efficient prefetcher is to strike a balance between performance and hardware overhead. Some state-of-the-art prefetchers achieve very high performance at the price of a very large storage requirement, which makes them not amenable to hardware implementations in commercial processors.**

We argue that merging memory access patterns can be a feasible solution to reducing storage overhead while obtaining high performance, although no existing prefetchers, to the best of our knowledge, have succeeded in doing so because of the difficulty of designing an effective merging strategy. After analysis of a large number of patterns, we find that the address offset of the first access in a certain memory region is a good feature for clustering highly similar patterns. Based on this observation, we propose a novel hardware data prefetcher, named Pattern Merging Prefetcher (PMP), which achieves high performance at a low cost. The storage requirement for storing patterns is largely reduced and, at the same time, the prefetch accuracy is guaranteed by merging similar patterns in the training process. In the prefetching process, a strategy based on access frequencies of prefetch candidates is applied to accurately extract prefetch targets from merged patterns. According to the experimental results on a wide range of various workloads, PMP outperforms the enhanced Bingo by 2.6% with 30× lesser storage overhead and Pythia by 8.2% with 6× lesser storage overhead.

***Keywords*-cache; hardware data prefetching;**

I. Introduction

Prefetching is one of the well-known techniques to speed up long-latency memory accesses for decades. With the ever-increasing memory consumption of modern applications, a cache hierarchy with limited capacity can be a bottleneck in performance improvements of processors [1]. Because caches can be effectively utilized in reducing memory access latency for reusable data, a large cache might be used to improve performance further. However, larger caches can lead to longer cache access latency. The performance improvement is limited when enlarging the capacity of high-level caches in the hierarchy (e.g., L1 Data Cache, L1D) due to their strict latency requirements. Prefetchers are often employed to reduce memory access latency by prefetching data in need without requiring a big on-chip area, leading to better cost performance than enlarging caches.

To accurately capture memory access patterns, some state-of-the-art prefetchers require more and more storage in recording memory access history [2]–[8]. The large storage requirement comes from two aspects. First, unlike a simple memory access pattern that can be described by a constant stride, a complex memory access pattern requires varied strides for description. It can be a considerable cost in recording complex patterns through bit vectors [9], delta sequences [10], etc. Second, many features, including Program Counters (PCs), memory addresses, memory address offsets, memory address strides, etc, and their combinations can be leveraged to index patterns for improving prefetch accuracy. A prefetcher for high performance tends to apply fine-grained features with large value ranges to index thousands of patterns, resulting in great storage consumption.

The art of designing an efficient prefetcher, dealing with complex memory access patterns, is to strike a balance between performance and storage overhead. We find that the major portion of the storage consumption of some prefetchers is caused by severe data redundancy. For instance, we find 82.9% of patterns are redundant in Bingo [2], [11], while astonishingly 24.2% of valid entries are allocated to the same pattern. The low storage efficiency due to the high data redundancy is the main reason for Bingo using a large table with $16,000$ entries.

Through analyzing a large number of memory access patterns captured from 125 traces, we observe that the patterns are highly similar, if they are indexed by the same *address offset of the first access* (named *Trigger Offset*) in a certain memory region. The access is named *Trigger Access* in the region. Selecting trigger offsets as features offers a great promise of designing an efficient pattern merging strategy, because the information loss is relatively small when merging similar patterns. As result, a prefetcher can consume less storage and obtain high performance if the patterns with the same trigger offset are merged. Because the patterns indexed by a trigger offset are not necessarily identical, the strategy must be deliberately designed such that characteristics of patterns can be maintained after merging, and prefetch targets can be efficiently extracted from merged patterns.

In this paper, we design a pattern merging strategy that can efficiently quantify characteristics of patterns. Moreover, we design a strategy that can accurately extract prefetch targets from merged patterns based on access frequencies of prefetch candidates. In a nutshell, we propose a

978-1-6654-6272-3/22/$31.00 ©2022 IEEE
DOI 10.1109/MICRO56248.2022.00071

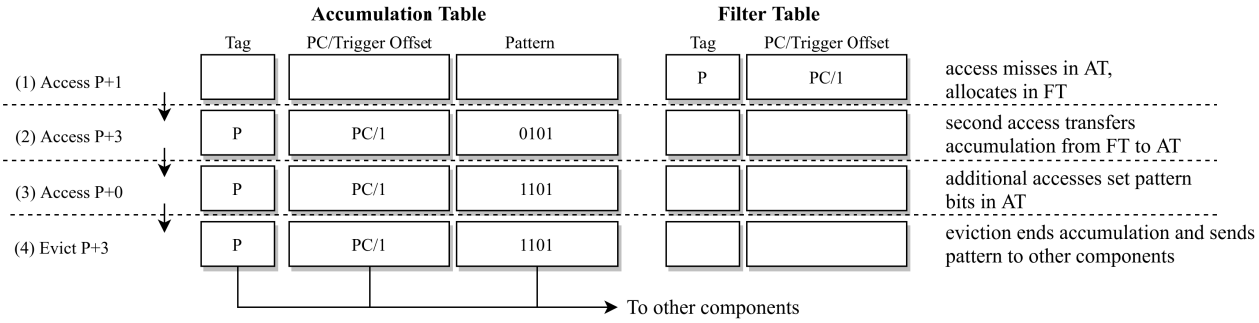


Figure 1: Pattern capturing procedure of SMS.

novel prefetcher, named Pattern Merging Prefetcher (PMP). Through exhaustive experiments on 125 traces from different benchmarks, PMP outperforms the enhanced Bingo by 2.6% with 30× lesser storage overhead and Pythia [7] by 8.2% with 6× lesser storage overhead in a single-core system.

We make the following contributions in this paper:

- We make a crucial observation that the memory access patterns are highly similar if they have the same trigger offset, after detailed analysis of a large number of memory access patterns captured from 125 traces.
- We propose a novel prefetcher, named Pattern Merging Prefetcher (PMP), including: a pattern merging strategy that quantifies characteristics of patterns and reduces the storage consumption; an extraction strategy based on access frequencies of prefetch candidates from merged patterns for accurate prefetching; an optimization using a dual pattern table structure to provide multi-feature-based pattern prediction.
- We show that PMP obtains better performance at a low storage cost compared to four state-of-the-art prefetchers over 125 traces by extensive experiments.

The remaining sections are organized as follows. Section II provides the background. Section III presents our motivations, i.e., three key observations of memory access patterns. Section IV describes the innovative mechanisms of PMP. An exhaustive performance evaluation is presented in Section V. Section VI discusses related work about hardware data prefetching with different pattern forms. Finally, the paper is concluded in Section VII.

II. Background

We first briefly introduce some typical prefetchers based on the bit vector pattern form. Next, we delve into the bit vector pattern capturing framework of Spatial Memory Streaming (SMS) [9], on which our prefetcher is based.

A. Prefetchers based on Bit Vectors

A bit vector describes the accessed positions in a memory region, each bit of which corresponds to an offset in the region. For example, the bit vector 1011 means that P, $P+2$, and $P+3$ have been accessed in memory region P. The bit vector form is first leveraged in SMS, whose efficiency has been proven for addressing complex memory access patterns. Bulk Memory Access Prediction and Streaming (BuMP) [12] improves SMS by reducing memory energy consumption. Bingo [2] improves SMS by using multiple features, combining PCs with offsets or addresses, to accurately locate patterns to be prefetched. Dual Spatial Pattern Prefetcher (DSPatch) [13] records memory access patterns with dual bit vectors, generated by an OR operation and an AND operation respectively, and uses different prefetch policies according to the bandwidth of environment.

The bit vector form has many advantages. First, any memory access distribution in a memory region can be represented with a bit vector. Second, bit vectors can represent dozens of offsets in a memory region at a very low cost. Benefiting from the high information density of bit vectors, a prefetcher can immediately generate many prefetch targets, leading to deep prefetching. Though bit vectors do not maintain temporal information of memory accesses, i.e., access order, a prefetcher can apply some heuristic methods to compensate for this drawback. For example, a prefetcher can prefetch the nearest target to the currently accessed address in advance. For a prefetcher based on bit vectors, the performance improvement, gained by attaching temporal information, is not significant [14].

B. Pattern Capturing Framework of SMS

SMS adopts a lightweight pattern capturing framework, which accounts for 2% of its total storage. It consists of two set-associative tables: the Filter Table (FT) and the Accumulation Table (AT). The FT records information of the first access to each memory region. The AT accumulates memory access patterns for each memory region. The pattern capturing procedure of SMS is shown in Fig. 1. First, when the memory region of a new memory access misses in the AT and the FT, the FT will allocate a new entry to store the PC and the address of the access. The offset of its address is a trigger offset. Second, when another access to the same region comes with a different offset, a bit vector is assembled with the offsets of these two accesses and sent to the AT.

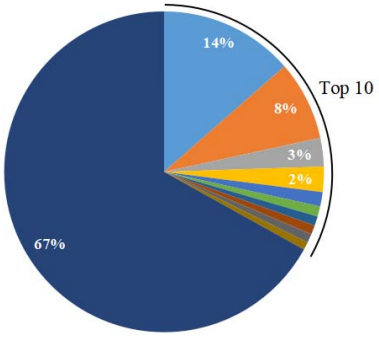


Figure 2: Percentages of top 10 frequent patterns.

Third, the offsets of the following accesses that belong to the region are used to update the bit vector in the AT. Finally, the accumulation process of the bit vector finishes when any cached data belonging to this region is evicted. The bit vector is sent to other components such as a pattern table.

III. Motivation

To analyze the characteristics of memory access patterns, we use the framework of SMS to capture patterns in 125 traces from SPEC CPU 2006 [15], SPEC CPU 2017 [16], PARSEC [17], and Ligra [18]. The FT is 4×16 set-associative, the AT is 8×16 set-associative, and the pattern length is 64, matching with cachelines in 4KB pages.

Observation 1: Only a tiny minority of memory access patterns occur with high frequency.

Because a bit vector consists of dozens of bits (64b), the huge status space (2^{64}) might introduce a large amount of data that no caches can afford. Fortunately, majority patterns occur infrequently. According to our experiments, 6.5×10^6 distinct patterns totally occur about 1.1×10^8 times among 125 traces, and 75.6% of distinct patterns appear only once. As shown in Fig. 2, the top 10 frequent patterns account for 33.1% of the total occurrences. Note that these patterns cover an extremely small portion ($1.55 \times 10^{-4}\%$) of distinct patterns. Moreover, the top 100 and the top 1000 frequent patterns account for 57.4% and 73.8% of the total occurrences respectively. Because only a tiny minority of patterns occur intensively, it is feasible to design a lightweight prefetcher with dozens of entries, in which only highly frequent patterns are maintained.

Observation 2: Indexing memory access patterns with fine-grained features can lead to severe data redundancy.

The indexing schemes used by state-of-the-art prefetchers [2]–[4], [9], [19] can hardly guarantee that the indexed memory access patterns are unique in their storage. As result, a large amount of data redundancy might be introduced. To quantitatively analyze the data redundancy of various indexing schemes using different features, we define the *Pattern Collision Rate* (PCR) as the number of distinct patterns related to a feature value and the *Pattern Duplicate Rate* (PDR) as the number of feature values related to a pattern. Fig. 3 shows examples of pattern collisions and pattern duplicates. We say the pattern 1101 has two duplicates because the feature values A and B both index it, and the patterns 1101 and 0101 collide because they are both indexed by the same feature value B. The PDR of 1101 is 2 and the PCR related to B is 2. A greater PDR indicates higher data redundancy, since the same pattern related to different feature values must be stored in multiple entries in a classical set-associative cache. The averages of the two metrics corresponding to various features are shown in Table I after analyzing 125 traces.

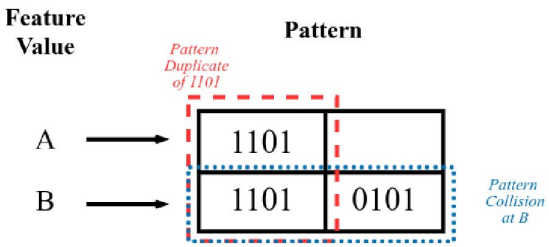


Figure 3: Pattern collision and pattern duplicate.

Table I: Average Pattern Collision/Duplicate Rates

Feature	Pattern Collision Rate	Pattern Duplicate Rate
PC (32b)	3823.6	2.2
Trigger Offset (6b)	2094.2	2.6
PC+Trigger Offset (38b)	269.0	6.3
Address (48b)	1.8	556.3
PC+Address (80b)	1.7	608.7

Intuitively, prefetchers tend to use fine-grained features with high bit widths, so that reduced PCRs can be obtained to effectively differentiate patterns. According to Table I, the 80-bit PC+Address feature has a PCR of 1.7, providing the highest resolution in recognizing patterns compared to the other listed features. However, large storage could be wasted for storing redundant patterns, because the PDR of the PC+Address feature is high (608.7). By evaluating Bingo that uses the PC+Address feature, 82.9% of its patterns are redundant at the end of simulation, and 24.2% of valid entries are occupied by the same pattern.

For features containing addresses, their high PDRs indicate that many patterns are shared among different memory regions. To decrease duplicate patterns caused by this reason, the high bits of addresses that represent regions cannot be used as a feature, so that the same patterns of different regions can be indexed into the same one. Because it is difficult to find a feature that can eliminate the duplication, we try to reduce duplication with features of low PDRs.

A new problem appears if we attempt to leverage features

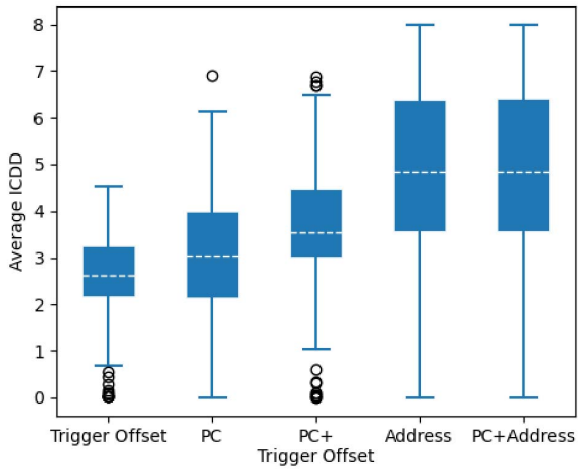
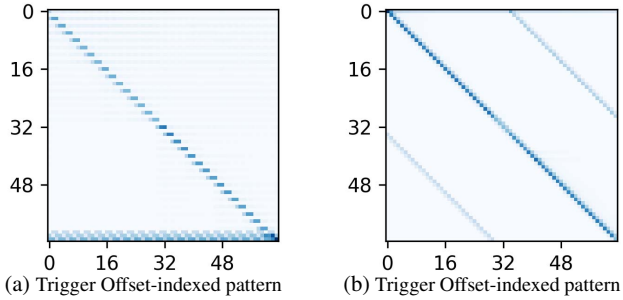
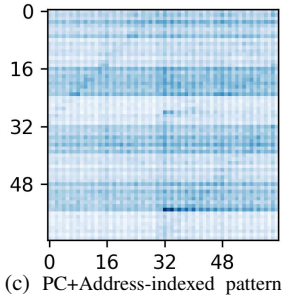


Figure 4: Box plot of average ICDDs. White dotted lines refer to means. Points refer to outliers.

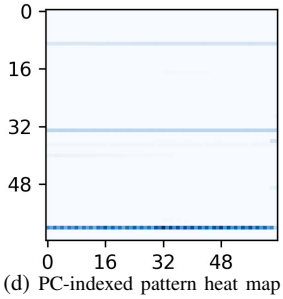


(a) Trigger Offset-indexed pattern heat map for a MCF trace.

(b) Trigger Offset-indexed pattern heat map for an Astar trace.



(c) PC+Address-indexed pattern heat map for a MCF trace.



(d) PC-indexed pattern heat map for a MCF trace.

Figure 5: Heat maps of patterns. The x-axis represents the accessed offsets (from 0 to 63) in 4KB pages. The y-axis represents the indexes (from 0 to 63) of each feature. A darker point means more patterns containing the offset.

with limited PDRs, such as the Trigger Offset feature, the PC feature, and the PC+Trigger Offset feature, to reduce data redundancy. According to their high PCRs, memory access patterns can be replaced frequently in a set-associative cache because thousands of distinct patterns inevitably collide with each other, resulting in poor prefetch accuracy. Even if we use an ideally large cache to save all the collided patterns, it is still difficult to select correct prefetch targets from them. As result, those features with small PDRs can hardly be employed in a naive manner to reduce data redundancy.

Observation 3: Patterns are highly similar if they have the same trigger offset.

To reduce overhead and, at the same time, keep the high performance of a prefetcher, a promising approach is to merge feature values or patterns. For example, data redundancy can be eliminated if feature values related to a pattern are merged into the same index. Challenges exist in implementing the idea of merging feature values or patterns for a prefetcher. On one side, we suppose that the feature values can be merged by a certain method such as an AND operation. The merged feature value keeps varying during the pattern capturing process, and so does the location of the related pattern in a cache. The requirement of frequently reallocating locations of patterns makes merging feature values infeasible. On the other side, we suppose that the patterns related to a feature value can be merged. Because the feature value is fixed, the location of the merged pattern in a cache is stable. But the merged pattern keeps varying during the pattern capturing process. In this case, how to guarantee prefetch accuracy for a prefetcher based on merged patterns becomes the key problem.

We hope to find a feature that can cluster similar patterns, thereby reducing the difficulty of designing an efficient approach for merging patterns. For example, the common memory access distribution of patterns can be kept after merging with a bit-wise AND operation, and the information loss caused by merging patterns with high similarity is relatively small. In addition, the common distribution of similar patterns can be a good prefetch prediction target. Because memory access patterns are presented as (bit) vectors, we can use the Intracluster Centroid Diameter Distance (ICDD), the double average distance between all of the vectors and the center of a cluster, to measure their similarity. A smaller ICDD indicates higher similarity of patterns in a cluster. The calculation formulas are:

$$ICDD(S) = 2\{\frac{\Sigma_{x \in S} d(x, \bar{V})}{|S|}\}. \\ \bar{V} = \frac{1}{|S|}\Sigma_{x \in S} x. \tag{1}$$

where S is a vector cluster, x is a vector belonging to S, and $d(\alpha, \beta)$ is the Euclidean distance between two vectors.

All the features have the same value range in our analysis. The Trigger Offset, hashed PC, hashed PC+Trigger Offset, hashed Address, and hashed PC+Address features all have a width of 6 bits, i.e., patterns are clustered into 64 sets (clusters). The average ICDD of 64 clusters is used to characterize the similarity of all clustered patterns. Fig. 4 illustrates the layouts of average ICDDs among 125 traces for each feature. The patterns with the same trigger offset

have the highest similarity. Trigger Offset can be a good feature for implementing an efficient pattern merging method.

To show clustered patterns straightforwardly, we draw patterns in several typical traces as heat maps. A point in a heat map represents the magnitude of occurrences of patterns that contain the corresponding offset. Fig. 5a is a heat map that shows the representative memory access distributions of a MCF trace. For most of memory accesses in the trace, the program tends to access a few positions around their current access addresses. These frequently visited positions form a blue dotted slash in Fig. 5a. When certain big trigger offsets appear, the program tends to issue backward memory accesses, which form three horizontal dotted lines at the bottom. Fig. 5b shows another memory access distribution from an Astar trace, in which the patterns can be described through three slashes. This indicates that the memory accesses of Astar obey a constant stride pattern.

However, the representative patterns cannot be observed when the hashed PC+Address feature is used. Fig. 5c shows the memory access distributions of the MCF trace in this situation. As the figure illustrates, patterns are scattered into all 64 sets. We can not tell the common memory access distributions in the heat map. The common characteristics of memory access patterns are destroyed, probably resulting in inaccurate merging results.

Moreover, because the PDR of the PC feature is the smallest in Table I, we would have thought that it could be a better choice compared to the other features. Surprisingly, we find the similarity of patterns indexed by PCs is lower than the Trigger Offset feature according to Fig. 4. For most traces, patterns clustered by the PC feature present overlapped memory access distributions in each set, leading to limited recognition of patterns. In addition, PCs generally distribute patterns into several concentrated sets, which can be leveraged to predict whether or not patterns can be prefetched. As shown in Fig. 5d, those PC-indexed patterns are allocated into several sets, forming horizontal lines. Finally, the PC feature is used to help predict prefetch levels as described later in Section IV-C.

Discussion. We think one major reason for Observation 3 is that Trigger Offset can be a more general feature relating to memory access behaviors of programs compared to PCs and addresses. For example, the backward memory accesses, as shown at the bottom of Fig. 5a, can be generated by the two loops of the following codes in MCF [20]:

```
// File: pflowup.c
// Method: MCF_primal_update_flow
// data are stored in a big array
for(;iplus!=w;iplus=iplus->pred)
{...}
for(;jplus!=w;jplus=jplus->pred)
{...}
```

Features containing addresses fail to cluster the same patterns because the accesses to a big array can cross many memory regions. The patterns can also hardly be clustered by the PC feature due to the different PCs of the two loops. In contrast, the trigger offsets can be a general and recognizable feature. First, the backward accesses to a big array will first load data from the end of a region, probably resulting in the same big trigger offset for the same pattern in different regions. Second, three major patterns generated by the loops can be differentiated by different big trigger offsets as Fig. 5a shows. They could be mixed if the PCs were used as a feature.

IV. Design: PMP

The prefetching mechanism of PMP consists of two processes working in parallel: the training process and the prefetching process. In this section, we first introduce a Pattern Merging strategy in the training process, enabling a substantial reduction of storage requirements without sacrificing performance. Second, we present a Prefetch Pattern Extraction strategy that generates highly accurate prefetch targets from merged patterns in the prefetching process. Third, we propose an optimized structure, Dual Pattern Tables, to leverage multiple features for improving prefetch accuracy further. Arbitration rules are also proposed to decide the final prefetch targets based on the predictions of the dual pattern tables. Then, we put it together and summarize the main flows. Finally, we discuss the overhead.

A. Pattern Merging

The observations in Section III inspire us to build a storage efficient prefetcher through merging memory access patterns clustered by trigger offsets. How to merge these patterns is a key problem that directly determines the effectiveness of our prefetcher. A bit-wise OR operation or a bit-wise AND operation could be an option for pattern merging, but the two operations are abandoned eventually. The OR operation creates a superset of patterns, e.g., the union of pattern 1111 and any other pattern is always 1111. The AND operation generates a common subset of patterns, e.g., the intersection of pattern 0000 and any other pattern is always 0000. As demonstrated in the examples above, a few outlier samples can obscure the differences in memory access patterns completely, leading to inaccurate records.

Based on extensive studies, we choose to merge bit vector patterns by counting the number of occurrences of each offset in bit vectors, instead of the two operations mentioned above. To achieve this goal, we apply a vector of counters (named *Counter Vector*), in which each element records the number of accesses to an offset, to merge patterns in a cluster. In the training process of PMP, the counters for offsets in a vector increase in parallel if the corresponding offsets are accessed in a bit vector pattern to be merged.

Fig. 6a illustrates an example of merging the bit vector $(0, 1, 1, 0, 1, 0, 0, 0)$, captured from the access sequence

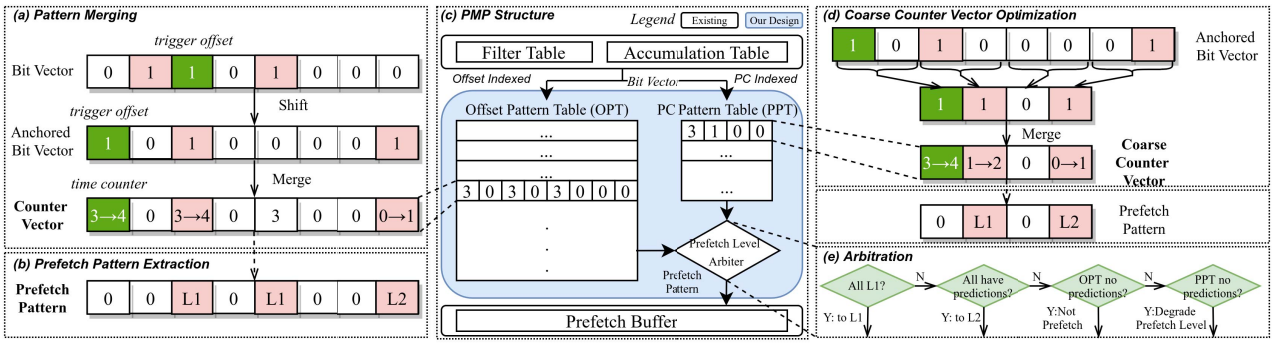


Figure 6: Architecture of PMP.

$P+2, P+1, P+4$ in memory region P, into the counter vector $(3,0,3,0,3,0,0,0)$. First, the bit vector needs shifting into an anchored bit vector. The bit vector is converted into $(1,0,1,0,0,0,0,1)$ by left circular shifting 2 positions because the trigger offset is 2. Then, the first, the third, and the last elements of the counter vector increase by 1 respectively. Finally, the counter vector is updated to $(4,0,4,0,3,0,0,1)$. Note that the counter corresponding to the trigger offset will always be the first element of a shifted vector and increases in every merging operation, so it is called *Time Counter*.

Because all the patterns are merged, old records cannot be simply evicted during the training process. Instead, all the elements in a counter vector are halved when the time counter saturates. This mechanism aims at reducing but keeping the effects of old records in the prefetching process. Supposing that the maximum value of time counters is 3, the counter vector $(4,0,4,0,3,0,0,1)$ is saturated in the example above, then it is halved to $(2,0,2,0,1,0,0,0)$.

The pattern merging strategy is efficient for two reasons. First, because the patterns to be merged have high similarity, their common characteristics remain after merging, which lays the groundwork for high prediction accuracy in the prefetching process. Otherwise, the merged patterns would be inevitably ambiguous for prefetching, if the patterns being merged were irrelevant. Second, unlike the OR/AND operation, which accepts/rejects all the differences in patterns, our strategy quantifies characteristics of patterns and reduces the storage consumption. The statistical results after merging can be leveraged for precisely predicting prefetch targets, which is described in the next subsection.

B. Prefetch Pattern Extraction

In the prefetching process, stored patterns can be triggered when a trigger access comes. Triggered patterns in the form of counter vectors cannot be simply replayed for prefetching like prior prefetchers do. Therefore, a conversion from triggered patterns to prefetch targets is required. Our strategy is to individually select target offsets by examining the corresponding elements in the triggered counter vector to form a *Prefetch Pattern*, which is a vector of target cache levels for each offset. We consider three different schemes.

Access-Number-based Extraction. A prefetch target for an offset is generated if the corresponding element of the triggered counter vector is equal to or greater than a prefetch threshold. We call this scheme Access-Number-based Extraction (ANE). For example, the counter vector $(4,2,0,1)$ can be converted to the prefetch pattern $(0, L1, 0, L1)$ if the prefetch threshold for L1D is 1, then $A+1$ and $A+3$ are the prefetch targets for the current cacheline address A. Please note that the trigger offset (the first offset after shifting) itself will never be prefetched. This scheme is easy to implement, but the obvious drawback is that any target offset needs an inevitable cold start time to reach the prefetch threshold. For a threshold T, an offset will not be prefetched until it has been visited T times, losing many prefetch chances.

Access-Ratio-based Extraction. For every element of the triggered counter vector, a ratio of it to the sum of all counters can be compared to a threshold for generating prefetch targets. We briefly call this scheme Access-Ratio-based Extraction (ARE). For example, given a prefetch threshold $1/4$ for L1D, the ratios of each element in the counter vector $(4,2,0,1)$ are $(?, 2/3, 0, 1/3)$. Because the trigger offset is excluded, its ratio is ignored. Then, the counter vector can be converted to the prefetch pattern $(0, L1, 0, L1)$. Then, $A+1$ and $A+3$ are the prefetch targets for the current cacheline address A. Though a counter vector is halved when the time counter saturates, the ratios can hardly be changed.[1] The ARE avoids retraining after halving if the pattern of following memory accesses does not vary, so the prefetching process continues. However, the ARE implicitly limits the maximum number of prefetches in one prediction, namely the prefetch depth. Because a prefetch target on an offset is generated only if its corresponding ratio is equal to or greater than a threshold, at most d offsets can

[1]The values of counters in a counter vector decrease to half, so does their sum. Therefore, the ratios will not change theoretically. However, the calculation results may be slightly affected because of precision limitations in hardware.

be prefetched for the prefetch threshold $1/d$. For a triggered vector containing 64 counters with the same value, e.g., a stream pattern, no addresses can be prefetched unless the threshold is lower than $1/63$ (exclude the trigger offset). But a low threshold that is smaller than $1/63$ may lead to inaccurate prefetching in most cases. The ARE introduces an unnecessary trade-off between the prefetch depth and the prefetch accuracy.

Access-Frequency-based Extraction. Access frequencies of offsets can be a good criterion for prefetch target selection. An access frequency is different from an access ratio that the former indicates how many times an offset occurs in a period, and it is not influenced by the occurrences of other offsets. Moreover, the access frequency of an offset can be easily calculated by dividing its counter by the time counter. The Access-Frequency-based Extraction (AFE) generates prefetch targets by comparing access frequencies of offsets to a prefetch threshold. For example, given an L1D prefetch threshold $1/4$, the frequencies of each counter in the counter vector $(4, 2, 0, 1)$ are $(?, 2/4, 0, 1/4)$, so the counter vector can be converted to the prefetch pattern $(0, L1, 0, L1)$, then $A+1$ and $A+3$ are the prefetch targets for the current cacheline address A. This scheme has several advantages. First, because the halving mechanism hardly affects the frequencies of offsets, the AFE avoids the extra retraining process when the following memory access patterns do not change. More importantly, the AFE has better adaptability to different kinds of patterns compared to the former two schemes. On one side, the AFE does not have the cold start problem. If an offset appears every time in the last T patterns, its frequency is 100% from the beginning of training which exceeds any threshold. This is friendly for patterns with a few repetitions. On the other side, no implicit restrictions are introduced by the AFE, since every offset that frequently occurs can be independently selected. This is friendly for stream-like patterns. Finally, we use the AFE as the default prefetch pattern extraction scheme.

To reduce the cache pollution in high-level caches without losing prefetch chances, PMP prefetches data into different cache levels depending on various thresholds. In the default configuration using the AFE, the confidence refers to frequencies. The threshold T_{l1d} for prefetching data into L1D is 50% and the threshold T_{l2c} for L2 Cache (L2C) is 15%. The target addresses, assembled using the current cacheline address and offsets with confidence greater than or equal to T_{l1d}, are prefetched to L1D. The targets with confidence greater than or equal to T_{l2c} but less than T_{l1d} are prefetched into L2C for reducing the risk of cache pollution in L1D. Fig. 6b shows an example that the prefetch pattern is $(0, 0, L1, 0, L1, 0, 0, L2)$ extracted from the counter vector $(4, 0, 4, 0, 3, 0, 0, 1)$.

As shown at the bottom of Fig. 6c, a new prefetch pattern is stored in the Prefetch Buffer (PB) and indexed by the region address of the trigger access. There is no fixed prefetch degree for PMP. When free Prefetch Queue (PQ) entries exist, PMP first assembles addresses using valid offsets in the prefetch pattern that are near the cacheline address of the trigger access and issues them to the corresponding cache levels. If PQ is full, the prefetching process is suspended. When any load with the address of the same region reappears and free PQ entries exist, the process continues with the prefetch pattern in the PB. Please note that prefetch requests are prohibited from occupying all MSHRs, at least one MSHR is remained for normal load/store requests.

C. Multi-Feature-based Prediction

Based on Observation 3, we notice that the PC feature of the trigger access can also be leveraged to help predict prefetch targets. The combination of PCs and trigger offsets can be used as a feature like prior research does, but it may not be the best option. First, the concatenated feature expands the index range greatly, necessitating a large table to store merged patterns. Second, patterns clustered by the combination of PCs and trigger offsets have lower similarity. The memory access patterns indexed by trigger offsets are separated into different sets again by PCs, bringing a greater divergence of patterns in a set than using the Trigger Offset feature or the PC feature alone.

Dual Pattern Tables. To enable multi-feature-based prediction and reduce the divergence of memory access patterns in a cluster, dual pattern tables are applied to maintain merged patterns indexed by trigger offsets and PCs respectively as Fig. 6c illustrates. Because counter vectors collect all the patterns indexed by the same feature value without evicting, the vectors can be stored in tagless direct-mapped tables. The Offset Pattern Table (OPT) indexed with trigger offsets is the primary pattern table, and the PC Pattern Table (PPT) indexed with PCs is the supplement pattern table. During the training process, the dual pattern tables are updated simultaneously. In the prefetching process, the candidate prefetch patterns are independently predicted by the two tables when a trigger access comes. Two candidate prefetch patterns are given by the tables using the AFE described in the last subsection.

Arbitration. An arbiter is applied to decide the final prefetch pattern depending on the predictions from the two tables. Though both pattern tables can give prefetch targets individually, it is better to discard the targets given by the PPT that are not included in the targets given by the OPT. The predictions of the OPT are more accurate than the PPT according to the experimental result in Section V-E3. As shown in Fig. 6e, the arbitration rules are as follows: 1) prefetches aiming at the same target offset are issued to L1D only if both pattern tables predict to prefetch data into L1D; 2) if the same target offset is predicted by both tables in which one table predicts to prefetch data into L2C, the target will be prefetched into L2C; 3) if the PPT has no predictions, the cache level of prefetches predicted by the OPT will be

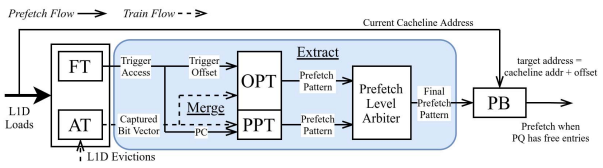


Figure 7: Training and prefetching flows of PMP.

Table II: Preset Parameters

Parameter	Value
OPT Counter Size	5b
PPT Counter Size	5b
OPT Pattern Length	64
PPT Pattern Length	32
Region Size	4KB
Monitoring Range	2
L1D Prefetch Threshold (T_{l1d})	50%
L2C Prefetch Threshold (T_{l2c})	15%

downgraded (e.g., L2C to Last Level Cache, LLC); 4) no prefetches will be issued if there are no predictions from the OPT. Please note that the final prefetch pattern after arbitration is stored in the PB.

Coarse Counter Vector. Because the predictions from the PPT only affect prefetch cache levels, the storage costs can be reduced further by monitoring several offsets with one counter. The number of offsets monitored by a counter is called *Monitoring Range*. As Fig. 6d depicts, the union of every several near bits of a bit vector is counted by a shorter counter vector, named *Coarse Counter Vector*. The 8-bit vector 10100001 is reduced to 1101 by joining every two bits, then 1101 is merged into the coarse counter vector $(3, 1, 0, 1)$. Every element in a coarse counter vector controls whether/where to prefetch on adjacent offsets at the prefetch level arbitration step. The final prefetch pattern in the example of Fig. 6 is $(0, 0, L1, 0, L2, 0, 0, L2)$ based on the two candidate prefetch patterns: $(0, 0, L1, 0, L1, 0, 0, L2)$ from the OPT and $(0, L1, 0, L2)$ from the PPT.

D. Putting It Together

Fig. 7 shows the training and the prefetching flows of PMP. The training process performs on L1D loads. If the region of an L1D load misses in the AT and the FT, it is a trigger access in the region. The pattern capturing framework records the following accesses in the region as described in Section II-B and the captured pattern is merged in the OPT and the PPT. The prefetching process performs at the same time as the trigger access comes. The offset and the PC of the trigger access are used to trigger (index) the OPT and the PPT respectively. The final prefetch pattern is obtained after the extraction and the arbitration, and it is then stored in the PB for prefetching.

Table III: Detailed Storage Overhead

Structure	Entry	Field	Storage
Filter Table	8×8	Region Tag (33b) Hashed PC (5b) Trigger offset (6b) LRU (3b)	376Bytes
Accumulation Table	2×16	Region Tag (35b) Hashed PC (5b) Trigger offset (6b) Bit Vector (64b) LRU (4b)	456Bytes
Offset Pattern Table	64×1	Counter Vector (320b)	2560Bytes
PC Pattern Table	32×1	Coarse Counter Vector (160b)	640Bytes
Prefetch Buffer	1×16	Region Tag (36b) Prefetch Pattern (126 bits) LRU (4b)	332Bytes
$Total \approx 4.3KB$			

E. Overhead

Table II lists all the preset parameters of PMP. Table III lists the overhead details. In our default configuration, the OPT indexed with trigger offsets contains 64 entries, and the PPT indexed with PCs has 32 entries. The FT and the AT have 64 and 32 entries respectively. The PB can store 16 prefetch patterns. Two bits are enough for four states of every offset: No Prefetch, Prefetch to L1D, Prefetch to L2C, and Prefetch to LLC. Therefore, a prefetch pattern requires 126 bits for 63 targets. The size of memory regions that each pattern corresponds to is 4KB. Finally, the total hardware overhead of PMP is 4.3KB (30× lower than Bingo).

We use CACTI [21] with its 22nm configuration to estimate the area consumption and the cache access time of our design. The area of the dual pattern table structure is 0.0069 mm^2. It is 151× smaller than the large set-associative pattern table in Bingo, which costs 1.0372 mm^2. The total access time (input and output) of the dual pattern table structure is 0.1ns, which is 11× shorter than the access time of Bingo's pattern table.

V. Evaluation

A. Methodology

1) Configuration: We use the ChampSim [22] simulator to evaluate prefetchers. Table IV illustrates the system configuration of ChampSim. We compare PMP to four state-of-the-art prefetchers: DSPatch [13], Bingo [2], [11], SPP+PPF [4], [23], and Pythia [7]. For a fair comparison, all prefetchers are placed at L1D, and no helper prefetchers exist in the other cache levels. That is, the five prefetchers are all single-level in our evaluation. DSPatch is a lightweight bit-vector-based prefetcher that records patterns through an OR and an AND operations. The competition version of Bingo in the third Data Prefetching Championships (DPC) [11] is the most powerful single-level prefetcher according to our experiments with a couple of state-of-the-art prefetchers.

Table IV: Simulated System Configuration

Core	One to four cores, 4GHz, 4-wide, 352-entry ROB, 128-entry LQ, 72-entry SQ, 4KB Page
TLBs	64-entry ITLB, 64-entry DTLB, 1536-entry L2DTLB
L1I	32KB, 8-way, 32-entry PQ, 8-entry MSHR, 4 cycles
L1D	48KB, 12-way, 8-entry PQ, 16-entry MSHR, 5 cycles
L2C	512KB, 8-way, 16-entry PQ, 32-entry MSHR, 10 cycles
LLC	2MB to 8MB, 16-way, 32 to 128-entry PQ, 64 to 256-entry MSHR, 20 cycles, Inclusive
DRAM	4GB 1 channel (1-core), 8GB 2 channels (4-core), 3200 MT/sec

Table V: Prefetcher Overhead

DSPatch	Bingo	SPP+PPF	Pythia	PMP
3.6KB	127.8KB	48.4KB	25.5KB	4.3KB

We enhance it by doubling the size of its pattern table to match its original configuration [2]. SPP+PPF is a strong competitor in DPC-3 [23], leveraging nine different features. Pythia is a new type of prefetcher built on machine learning in hardware. Table V shows the storage overhead of these prefetchers.

2) Benchmarks: Over one hundred traces, captured from SPEC CPU 2006 [15], SPEC CPU 2017 [16], PARSEC [17], and Ligra [18], are used to evaluate the single-core performance of the five prefetchers. For SPEC CPU 2006 and SPEC CPU 2017, we use the instruction traces provided by DPC-2 and DPC-3 [24], [25]. For PARSEC and Ligra, we use the traces provided by Pythia [7]. All the traces have more than five LLC misses per kilo instructions (MPKI). Table VI lists the numbers of traces for each benchmark.

We evaluate the multi-core performance of the five prefetchers with homogeneous and heterogeneous multi-programmed workloads in a 4-core processor. For homogeneous workloads, we examine prefetchers with 125 traces, each of which is simultaneously performed by different cores. For heterogeneous workloads, we first classify traces into three classes: Low MPKI ($5 < \text{MPKI} \leq 10$), Medium MPKI ($10 < \text{MPKI} \leq 20$), and High MPKI ($\text{MPKI} > 20$). Then, we randomize traces according to their classes and generate workloads as Table VII shows. We also examine prefetchers with CloudSuite traces [26] but do not see many performance improvements, so the results are omitted.

In both single-core and multi-core systems, we use the first 50 million instructions of a trace to warm up micro-architectural structures and the next 200 million instructions to examine the performance of each core.

B. Single-core Performance

Fig. 8 illustrates the normalized IPCs (NIPCs) of the five prefetchers in the single-core system. We observe that PMP outperforms the other four prefetchers at a very low storage cost. PMP improves the performance of the non-prefetching baseline by 65.2% and outperforms DSPatch, Bingo, SPP+PPF and Pythia by 41.3% (up to 177.4%), 2.6% (up to 62.4%), 6.5% (up to 59.2%) and 8.2% (up to 183.1%). DSPatch requires the lowest storage among the five prefetchers. However, the low performance of DSPatch indicates that its pattern capturing strategies, an OR and an AND operations, are inefficient. Bingo is a heavy prefetcher that is $3\times$ larger than a typical L1D, so it is more realistic for it to be placed at low-level caches, which brings lower performance. PMP (at L1) outperforms the original Bingo at LLC by 16.5%. SPP+PPF leverages nine features for filtering predictions generated by an aggressive Signature Path Prefetcher (SPP) [10], which performs much better than the original SPP. Though SPP+PPF applies more features compared to Bingo and PMP, its performance is lower, which indicates that the number of features is not the more the better. In contrast to PMP that can issue dozens of prefetches in one prediction, Pythia only generates one prefetch target per prediction, which limits its prefetch depth and performance. Pythia cannot deeply prefetch due to its poor prefetch accuracy, which will be described in the next subsection.

Table VI: Traces

SPEC 06	SPEC 17	Ligra	PARSEC	Total
38	36	42	9	125

Table VII: Heterogeneous 4-core Workloads

MIX	#
All Low MPKI	10
All Medium MPKI	10
All High MPKI	10
Half Low and Half Medium MPKI	10
Half Low and Half High MPKI	10
Half Medium and Half High MPKI	10

On the desktop or scientific workloads such as SPEC CPU 2006 and SPEC CPU 2017, the performance of PMP is better than that on the other workloads, since the memory access patterns of these workloads are regular. Though prefetchers usually require large hardware storage to deal with irregular memory accesses generated by Ligra and PARSEC, PMP still outperforms the heavy prefetchers (Bingo, SPP+PPF and Pythia) at a lower cost on these traces.

C. Coverage & Accuracy in Single-core

Prefetch coverage and prefetch accuracy are two metrics that explain a lot about performance of prefetchers. We define the coverage as the ratio of reduced load misses to the total load misses of the baseline, and the accuracy as the ratio of useful prefetches to the sum of useful and useless prefetches. PMP, SPP+PPF, and Bingo prefetch data into multiple levels of caches directly: PMP can fill data into L1D, L2C, and LLC; SPP+PPF and Bingo can fill data into

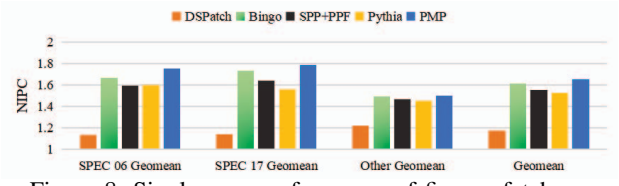


Figure 8: Single-core performance of five prefetchers.

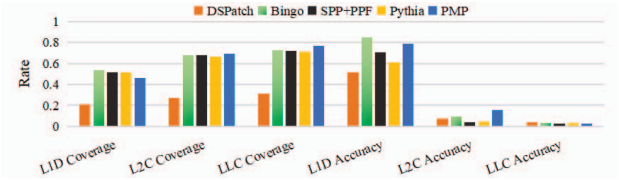


Figure 9: Coverage and accuracy of five prefetchers.

L1D and L2C based on their configuration. We examine coverage and accuracy on different cache levels separately.

Fig. 9 shows the coverage and the accuracy of these prefetchers. We observe that PMP obtains the highest L2C coverage and the highest LLC coverage among five prefetchers. Its L1D coverage is much higher than DSPatch by 121.1%, which is competitive compared to the other prefetchers. The L1D accuracy of PMP exceeds DSPatch, SPP+PPF and Pythia by 52.8%, 11.4% and 29.2%. The L2C accuracy of PMP exceeds DSPatch, Bingo, SPP+PPF, and Pythia by 118.1%, 66.8%, 302.3%, and 243.7%. The high accuracy indicates that the prediction strategy of PMP is accurate. Since the Trigger Offset and PC features allow patterns to be shared between different memory regions, prefetched data can be demanded even if it is not ever accessed before in a certain memory region, reducing compulsory misses. As result, the coverage of PMP is also high. In a nutshell, the high coverage and the high accuracy contribute much to the high performance of PMP.

The results show that the L2C accuracy and the LLC accuracy are much lower than the L1D accuracy for all the five prefetchers. This is because all the prefetchers train on L1D accesses. In the case of inclusive caches, prefetches for high-level caches will implicitly prefetch data to low-level caches for keeping inclusiveness. Since many load/store requests are invisible to low-level caches, the corresponding prefetch accuracy is lower.

Fig. 10 shows the average valid prefetches that break down into useful and useless prefetches in different cache levels. We observe that PMP obtains high performance by suppressing the cache pollution in L1D and prefetching speculatively for low-level caches. First, PMP effectively restricts useless prefetches in the storage-limited L1D, while still offering many useful prefetches. DSPatch, SPP+PPF, and Pythia fail to limit the cache pollution in L1D compared to PMP. Though SPP+PPF and Pythia generate more useful prefetches than PMP, their large numbers of L1D

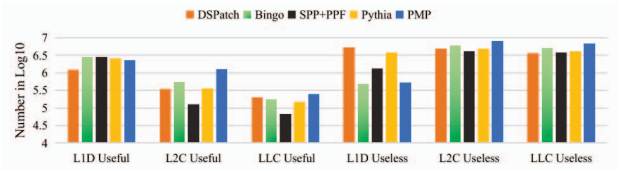


Figure 10: Average useful and useless prefetches.

useless prefetches hurt performance. Bingo issues many useful prefetches while producing the least amount of useless prefetches in L1D, benefiting from applying the fine-grained feature (PC+Address) of the greatest pattern differentiation ability among the five prefetchers as Table I shows. However, it also requires the largest storage space. Second, because PMP has a low L2C threshold (15%) and its first arbitration rule for prefetching data into L1D is strict, it speculatively prefetches data into low-level caches. Benefiting from the accurate prefetching strategy, the numbers of useful L2C and LLC prefetches of PMP are much larger than the other four prefetchers. Since L2C and LLC have larger storage space and are not latency-sensitive compared to L1D, it is affordable for them to contain more useless prefetches. The useless L2C/LLC prefetches of PMP do not hurt performance much.

D. Memory Traffic in Single-core

Prefetchers generally generate many memory access requests that consume additional memory bandwidths. To evaluate the memory traffic of the five prefetchers, we calculate the ratio of total memory access requests to the requests of the non-prefetching baseline as the *Normalized Memory Traffic* (NMT). Through evaluation, the NMTs of SPP+PPF and Pythia are 129.0% and 139.1%. Because triggered patterns in the bit vector form can generate dozens of prefetch targets in every prediction, the NMTs of DSPatch and Bingo are higher than the former two prefetchers, which are 159.8% and 164.2% respectively.

PMP produces the highest NMT (199.6%). It might be because PMP has the most aggressive prefetch policy compared to the other four prefetchers. First, counter vectors can generate up to 63 prefetch targets in one prediction, and prefetches will be issued if free PQ entries exist. Second, the prefetch condition of PMP is easy to meet because there are no tags for matching. After training PMP for a brief time, each trigger offset can issue a bundle of prefetches. PMP issues 1.46×10^7 prefetches per trace, which are 58.0% more than Bingo. This aggressive prefetch policy provides the deeper speculation through prefetching. Benefiting from the accurate prediction strategy, more useful prefetches are generated, so that PMP obtains the higher L2C/LLC prefetch coverage compared to other four prefetchers at the price of more memory traffic. Considering the high memory bandwidth consumption, high frequency memory (DDR4 [27], DDR5 [28], HBM [29], etc.) is appealing to PMP.

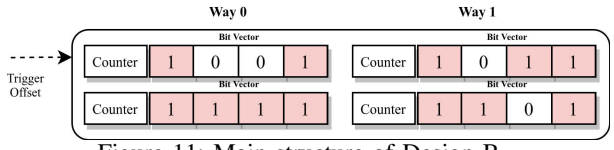


Figure 11: Main structure of Design B.

Table VIII: Performance of Design B

Ways	8	32	128	512
NIPC	1.176	1.188	1.215	1.224

To limit the NMT of PMP without introducing extra overhead, a straightforward approach is to control the aggressiveness of prefetches for low-level caches (L2C and LLC). When the prefetch degree for low-level caches is 1, PMP still outperforms Bingo by 1.3% with a lower NMT (159.0%).

E. Performance Analysis in Single-core

1) Pattern Merging Strategy: In Section V-B, the comparison between PMP and DSPatch has indicated that it can be inefficient to merge patterns by an OR or an AND operation. Other than the merging strategy of PMP, it could be possible to obtain low storage consumption and good performance by merging the identical patterns only, considering spatial and temporal locality of workloads. A bit vector attached with a counter can be used to count repetitions of identical patterns, in which the counter can be used for target pattern selection, e.g., the ANE scheme can be used to determine the prefetch pattern according to the counter. Instead of individually selecting each offset, all the valid offsets in the bit vector with a counter that exceeds the threshold are the prefetch targets. The enhanced bit vectors can be stored in a set-associative cache indexed with trigger offsets. This design is named *Design B*, whose main structure is illustrated in Fig. 11. We compare Design B to the pattern merging strategy of PMP. Table VIII depicts that the performance of Design B grows as the associativity increases. However, PMP outperforms Design B with 512 ways by 34.9%. Our pattern merging strategy is more efficient and eliminates the impact of a large number of evictions. Bingo does not suffer from severe evictions because of using features with low PCRs.

2) Prefetch Pattern Extraction Schemes: We consider three schemes for the prefetch pattern extraction in Section IV-B. For the ANE, the T_{l1d} is 16 and the T_{l2c} is 5 to scale close to the default thresholds of the AFE. The thresholds are the same between the ARE and the AFE.

Comparing to the AFE, the ARE performs poorly, which improves the baseline by 5.0%. We find that the poor adaptability of the ARE for prefetching severely limits the prefetch coverage. Because stream patterns can hardly be extracted by the ARE, a large number of prefetch opportunities are lost. The performance of the ARE is very low on many traces that contain a large number of stream patterns. Furthermore, PMP obtains no performance improvements by tuning the thresholds of the ARE. Though the similar methods of the ARE work well in many prefetchers [4], [10], [30], the ARE is not suitable for our prefetcher.

Because the ANE has the cold start time problem, and the prefetching process could be interrupted by the halving mechanism, the performance of the ANE is slightly impacted. PMP using the ANE achieves a 60.3% improvement beyond the baseline, which is 2.9% lower than the AFE. The ANE can be another feasible scheme with reduced hardware complexity compared to the AFE.

3) Multi-Feature-based Prediction: We compare the performance of the dual pattern table structure to the single table indexed with the combination of PCs and trigger offsets. Because PMP uses 6-bit trigger offsets and 5-bit PCs, the number of entries storing patterns increases from 96 ($2^6 + 2^5$) to 2048 (2^{6+5}) when the combined feature is used. However, the performance degrades by 3.1% compared to the dual pattern table structure.

We also evaluate the performance of PMP with a single feature. The performance reduces by 2.4% when using a single OPT. Moreover, we attempt to use a single PPT with the same size as the OPT, and the performance reduces by 3.5%. We find that the single-feature-based PMP prefetches more data into higher-level caches than the multi-feature-based PMP because of a lack of the prefetch level arbitration. As result, more useless prefetches are produced. For example, PMP with the single PPT generates 38.3% more useless prefetches at L1D compared to the default PMP. The large number of useless prefetches at L1D hurt its performance.

4) Preset Parameters: **Pattern Length.** The length of counter vectors depends on the size of tracked memory regions. Given a 4KB memory region, the accesses to each

Table IX: Performance and Overhead of PMP Under Different Pattern Lengths

Pattern Length	Region Size	Overhead	NIPC
64	4KB	4.3KB	1.652
32	2KB	2.5KB	1.626
16	1KB	1.6KB	1.572

Table X: Performance of PMP Under Different Trigger Offset Widths and Counter Sizes

Trigger Offset Width (b)	NIPC	Counter Size (b)	NIPC
6	1.652	2	1.624
7	1.654	3	1.634
8	1.655	4	1.648
9	1.657	5	1.652
10	1.657	6	1.653
11	1.657	7	1.654
12	1.658	8	1.655

Table XI: Performance of PMP Under Different Pattern Monitoring Ranges

Monitoring Range	1	2	4	8
NIPC	1.650	1.652	1.630	1.615

cacheline inside the region can be tracked using a vector containing 64 counters. PMP does not support cross-page prefetching, so we only discuss memory regions smaller than 4KB pages. As Table IX shows, we evaluate the performance of PMP with different lengths of patterns at different scales (i.e., in 4KB, 2KB, and 1KB regions), namely PMP-64, PMP-32, and PMP-16. The results show that the performance degrades as the pattern length becomes shorter. Patterns can be folded in the case of shrinking the tracked memory regions, which impacts the statistical results of pattern merging, so that the accuracy of extracting the prefetch patterns is lower. According to our analysis, the L2C prefetch accuracy of PMP-32 decreases from 15.2% to 10.6% compared to PMP-64, and the L2C accuracy decreases to 8.0% after reducing the pattern length to 16.

Though the performance decreases, the overall overhead is greatly reduced when using short patterns. PMP-16 is still competitive in terms of performance improvements, which outperforms DSPatch by 34.5% at a 2× lesser storage cost. PMP-32 outperforms Bingo by 1.0% at a 51× lesser storage cost. We attempt to adjust the size of tracked memory regions of Bingo from 2KB to 4KB. But this adjustment does not affect the accuracy of Bingo because its prefetch accuracy relies on the fine-grained features.

Trigger Offset Width. Table X shows the NIPCs of PMP under different trigger offset widths. As the results show, the performance increases with the width of trigger offsets. To avoid hash collisions, the sizes of direct-mapped tables are equal to the value ranges of features. In this case, the storage overhead increases exponentially with the width of trigger offsets. Comparing to the default PMP, the overhead increases by 64× using 12-bit trigger offsets while the NIPC increases by only 0.4%. Since the gain of performance improvement is negligible, we finally choose to use 6-bit trigger offsets.

Counter Size. The counter sizes of the OPT and the PPT can be different. We set the counter size of the PPT to 5 bits and only modify the counter size of the OPT. As described in Section IV-A, by leveraging the halving mechanism to reduce the effects of old records, a larger counter size allows maintaining history for a longer period. Table X illustrates the performance of PMP under different counter sizes. With the increase of counter sizes, the performance can increase accordingly. The results show that the extraction strategy can make better predictions with longer history.

Monitoring Range. As mentioned in Section IV-C, we reduce the storage overhead of the PPT by monitoring

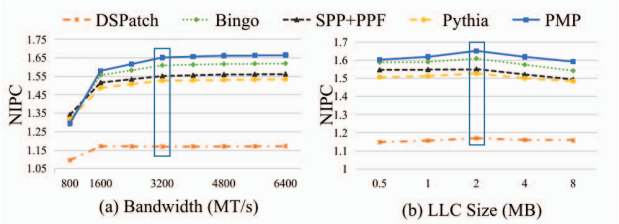


Figure 12: Performance of five prefetchers under different memory bandwidths and LLC sizes.

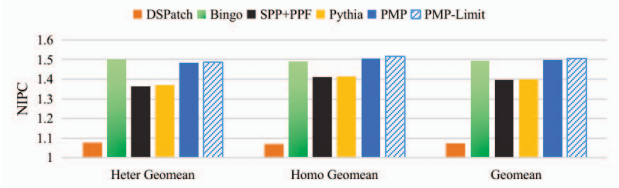


Figure 13: Multi-core performance.

several offsets with a counter. According to Table XI, the monitoring range 2 can be a good choice for reducing overhead while maintaining high performance. When the monitoring range is 8, the performance is almost the same as using a single OPT, which makes the dual pattern table structure useless.

F. Sensitivity in Single-core

1) Bandwidth: We examine the five prefetchers under various memory bandwidths as Fig. 12a depicts. PMP performs better than the other prefetchers when the bandwidth is greater than or equal to 1600 MT/s. When the bandwidth reaches 3200 MT/s, the performance is close to peak. PMP slightly underperforms Bingo, SPP+PPF and Pythia at the 800 MT/s bandwidth due to the greater bandwidth requirements, but it still outperforms DSPatch, applying various prefetch policies based on bandwidths, by 18.2%. Because the number of prefetches hardly varies under different memory bandwidths, the performance of PMP is limited when the bandwidth is low.

2) LLC Size: Fig. 12b shows the performance change of the five prefetchers under different LLC sizes. We enlarge the LLC by increasing the number of LLC sets. PMP outperforms the other four prefetchers in different LLC sizes. With the 8MB LLC setting, PMP outperforms Bingo by 3.3%. The performance gap between PMP and Bingo becomes larger when the LLC size increases because the impact of cache pollution caused by useless prefetches is reduced, which is friendly for aggressive prefetching.

G. Multi-core Performance

Fig. 13 shows the multi-core performance of the prefetchers. PMP-Limit is the PMP whose prefetch degree for low-level caches is 1. PMP outperforms DSPatch by 39.6%, and the two heavy prefetchers SPP+PPF and Pythia by 7.3% and

6.9% respectively. PMP can match the performance of Bingo on homogeneous and heterogeneous workloads. Because the aggressive prefetching strategy consumes a larger amount of bandwidth resources in the 4-core system, the performance improvement is slightly reduced compared to PMP in the single-core system. After limiting prefetching, PMP-Limit obtains 1% higher performance compared to Bingo.

VI. Related Work

A. Prefetchers based on Constant Strides

Some memory access patterns can be described using constant strides. A stride is the difference between two continuous access addresses. This is a straightforward description of memory access patterns, which is widely used by prefetchers. For example, Next Line Prefetcher (NL) [31] can be seen as a prefetcher that always prefetches data by one cacheline stride. Constant-stride-based prefetchers allow different strides for different memory regions [32]. Best Offset Prefetcher (BOP) [3] periodically calculates the confidence of different strides and chooses the best stride for prefetching. Sandbox Prefetcher (SP) [19] applies a similar method. The difference between BOP and SP is that the former records real memory accesses for calculating confidence and the latter uses a bloom filter to record fake prefetches instead. Though prefetchers built on constant strides can improve performance at very low storage costs, it can be hard to predict complex patterns. Because constant strides only assume that future accesses arrive at a certain pace, the patterns that consist of variable strides, like the address sequence $(P+1, P+2, P+4, P+3, P+1)$ in the memory region P, are beyond their capabilities.

B. Prefetchers based on Delta Sequences

Delta sequences use several deltas to describe the variation of strides for access sequences, e.g., the delta sequence for the address sequence $(P+1, P+2, P+4, P+3, P+1)$ in the memory region P is $(1, 2, -1, -2)$. Different from the other pattern forms that require addition information for indexing, delta sequences of memory accesses can be utilized to index themselves, i.e., the prefixes of delta sequences can be used as a feature. For example, if the prior access history forms a delta sequence $(1, 2, -1)$, we can predict the next delta is -2 in the example above. Signature Path Prefetcher (SPP) [10] captures patterns using five-delta sequences. The first four deltas of a sequence are compressed as a signature to trigger the last delta. Variable Length Delta Prefetcher (VLDP) [33] uses separated tables to maintain delta sequences with different lengths. Matryoshka [30] supports variable-length sequence matching by coalescing variable-length sequences into a single table.

Delta sequences can record the spatial and temporal information of memory access patterns. An advantage of recording memory access patterns through delta sequences is that the common parts of sequences can be shared, resulting in a reduction of data redundancy. Moreover, no additional features are required for indexing the patterns described through delta sequences. However, prefetchers relying on delta sequences are not accurate enough. To address this obstacle, Perceptron-based Prefetch Filtering (SPP+PPF) [4] designs a heavy perceptron-based filter with nine features to improve the accuracy of SPP. Moreover, prefetchers based on delta sequences have to yield prefetch addresses step by step using a recursive look-ahead strategy [4], [5], [10], [30]. It is difficult for prefetchers, relying on delta sequences, to issue dozens of prefetches immediately like Bingo does for deep prefetching.

C. Other Hardware Data Prefetchers

Some other prefetchers pay attention to irregular memory access patterns such as Global History Buffer (GHB) [34], Irregular Stream Buffer (ISB) [35], Managed Irregular Stream Buffer (MISB) [6], and Triage [8]. Because irregular memory access patterns usually cross many pages, which are difficult to be described through pattern forms such as constant strides, delta sequences, or bit vectors. GHB uses a large circular history buffer to record access history. ISB reconstructs physical addresses into structural addresses and stores them in memory. MISB improves ISB by filtering unnecessary memory requests with bloom filters. Triage organizes patterns as key-value pairs of addresses and requires up to the half storage of a LLC for eliminating requirements of off-chip storage. Most of them require too much storage and are only effective in irregular situations, which makes their design unaffordable in general processors.

VII. Conclusion

In this paper, we design a low-overhead and powerful prefetcher, named Pattern Merging Prefetcher (PMP). By analyzing many features, we find that the Trigger Offset feature can be used to cluster memory access patterns with high similarity. By merging similar patterns, the storage overhead of our prefetcher is largely reduced, and an accurate prefetch pattern extraction strategy with various effective schemes is applied. Moreover, an optimized prediction mechanism based on multiple features is proposed. PMP outperforms the non-prefetching system by 65.2%, and exceeds the performance of enhanced Bingo by 2.6% with 30× lesser storage overhead and Pythia by 8.2% with 6× lesser storage overhead in a single-core system.

We believe that there is still room for further exploration in the design idea of pattern merging. In the future, we plan to do further research on fundamental reasons behind the three observations and try to apply the design idea to other predictors in processors for improving performance.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their feedback.

References

[1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *SIGARCH Comput. Archit. News*, vol. 23, no. 1, pp. 20–24, 1995. [Online]. Available: https://doi.org/10.1145/216585.216588

[2] M. Bakhshalipour, M. Shakerinava, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Bingo spatial data prefetcher," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2019, pp. 399–411.

[3] P. Michaud, "Best-offset hardware prefetching," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 469–480.

[4] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, "Perceptron-based prefetch filtering," in *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 1–13.

[5] P. Papaphilippou, P. H. J. Kelly, and W. Luk, "Pangloss: a novel markov chain prefetcher," *CoRR*, vol. abs/1906.00877, 2019. [Online]. Available: http://arxiv.org/abs/1906.00877

[6] H. Wu, K. Nathella, D. Sunwoo, A. Jain, and C. Lin, "Efficient metadata management for irregular data prefetching," in *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*, S. B. Manne, H. C. Hunter, and E. R. Altman, Eds. ACM, 2019, pp. 449–461. [Online]. Available: https://doi.org/10.1145/3307650.3322225

[7] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, *Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning*. New York, NY, USA: Association for Computing Machinery, 2021, p. 1121–1137. [Online]. Available: https://doi.org/10.1145/3466752.3480114

[8] H. Wu, K. Nathella, J. Pusdesris, D. Sunwoo, A. Jain, and C. Lin, "Temporal prefetching without the off-chip metadata," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*. ACM, 2019, pp. 996–1008. [Online]. Available: https://doi.org/10.1145/3352460.3358300

[9] S. Somogyi, T. F. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, "Spatial memory streaming," in *Proceedings of the 33rd Annual International Symposium on Computer Architecture*, ser. ISCA '06. USA: IEEE Computer Society, 2006, p. 252–263. [Online]. Available: https://doi.org/10.1109/ISCA.2006.38

[10] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti, "Path confidence based lookahead prefetching," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.

[11] M. Bakhshalipour, M. Shakerinava, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Accurately and maximally prefetching spatial data access patterns with bingo," 2019. [Online]. Available: https://dpc3.compas.cs.stonybrook.edu/pdfs/Accurately.pdf

[12] S. Volos, J. Picorel, B. Falsafi, and B. Grot, "Bump: Bulk memory access prediction and streaming," in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*, 2014, pp. 545–557.

[13] R. Bera, A. V. Nori, O. Mutlu, and S. Subramoney, "Dspatch: Dual spatial pattern prefetcher," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 531–544. [Online]. Available: https://doi.org/10.1145/3352460.3358325

[14] M. Sutherland, A. Kannan, and N. Enright Jerger, "Not quite my temp: Matching prefetches to memory access times," in *Data Prefetching Championship Workshop*, 2015.

[15] "Spec cpu 2006." [Online]. Available: https://www.spec.org/cpu2006/

[16] "Spec cpu 2017." [Online]. Available: https://www.spec.org/cpu2017/

[17] "Parsec." [Online]. Available: http://parsec.cs.princeton.edu/

[18] J. Shun and G. E. Blelloch, "Ligra: A lightweight graph processing framework for shared memory," in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 135–146. [Online]. Available: https://doi.org/10.1145/2442516.2442530

[19] S. H. Pugsley, Z. Chishti, C. Wilkerson, P. Chuang, R. L. Scott, A. Jaleel, S. Lu, K. Chow, and R. Balasubramonian, "Sandbox prefetching: Safe run-time evaluation of aggressive prefetchers," in *20th IEEE International Symposium on High Performance Computer Architecture, HPCA 2014, Orlando, FL, USA, February 15-19, 2014*. IEEE Computer Society, 2014, pp. 626–637. [Online]. Available: https://doi.org/10.1109/HPCA.2014.6835971

[20] "Mcf homepage." [Online]. Available: https://www.zib.de/opt-long_projects/Software/Mcf/

[21] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.

[22] "Champsim simulator." [Online]. Available: https://github.com/ChampSim/ChampSim

[23] E. Bhatia, G. Chacon, E. Teran, P. V. Gratz, and D. A. Jiménez, "Enhancing signature path prefetching with perceptron prefetch filtering," 2019. [Online]. Available: https://dpc3.compas.cs.stonybrook.edu/pdfs/Enhancing_signature.pdf

[24] "The 2nd data prefetching championship," 2015. [Online]. Available: http://comparch-conf.gatech.edu/dpc2/

[25] "The 3rd data prefetching championship," 2019. [Online]. Available: https://dpc3.compas.cs.stonybrook.edu/

[26] "Cloudsuite traces." [Online]. Available: https://www.dropbox.com/sh/pgmnzfr3hurlutq/AACciuebRwSAOzhJkmj5SEXBa/CRC2_trace?dl=0&subfolder_nav_tracking=1

[27] "Jedec-ddr4." [Online]. Available: https://www.jedec.org/sites/default/files/docs/JESD79-4.pdf

[28] "Jedec-ddr5." [Online]. Available: https://www.jedec.org/standards-documents/docs/jesd79-5a

[29] "Hbm specification." [Online]. Available: https://www.amd.com/Documents/High-Bandwidth-Memory-HBM.pdf

[30] S. Jiang, Y. Ci, Q. Yang, and M. Li, *Matryoshka: A Coalesced Delta Sequence Prefetcher*. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3472456.3473510

[31] A. J. Smith, "Sequential program prefetching in memory hierarchies," *Computer*, vol. 11, no. 12, pp. 7–21, 1978. [Online]. Available: https://doi.org/10.1109/C-M.1978.218016

[32] T. Chen and J. Baer, "Effective hardware based data prefetching for high-performance processors," *IEEE Trans. Computers*, vol. 44, no. 5, pp. 609–623, 1995. [Online]. Available: https://doi.org/10.1109/12.381947

[33] M. Shevgoor, S. Koladiya, R. Balasubramonian, C. Wilkerson, S. H. Pugsley, and Z. Chishti, "Efficiently prefetching complex address patterns," in *Proceedings of the 48th International Symposium on Microarchitecture, MICRO 2015, Waikiki, HI, USA, December 5-9, 2015*, M. Prvulovic, Ed. ACM, 2015, pp. 141–152. [Online]. Available: https://doi.org/10.1145/2830772.2830793

[34] K. J. Nesbit and J. E. Smith, "Data cache prefetching using a global history buffer," *IEEE Micro*, vol. 25, no. 1, pp. 90–97, 2005. [Online]. Available: https://doi.org/10.1109/MM.2005.6

[35] A. Jain and C. Lin, "Linearizing irregular memory accesses for improved correlated prefetching," in *The 46th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-46, Davis, CA, USA, December 7-11, 2013*, M. K. Farrens and C. Kozyrakis, Eds. ACM, 2013, pp. 247–259. [Online]. Available: https://doi.org/10.1145/2540708.2540730